

Math Nuggets for GEN AI

Reinforcement Learning Basics: Mathematical Foundations for Policy Optimization

By Dr. Anish Roychowdhury, Plaksha University

September 29, 2025

From Supervised Learning to Decision Making

While supervised learning excels at pattern recognition from fixed datasets, many AI applications require sequential decision-making under uncertainty. Reinforcement Learning (RL) provides a mathematical framework where agents learn optimal behavior through trial and error, maximizing cumulative rewards over time.

Consider the task: "Train an AI agent to play chess, learning from wins and losses rather than memorizing expert games."

Traditional supervised learning faces limitations for sequential decision-making:

- **No Sequential Context:** Each prediction is independent, ignoring long-term consequences
- **Fixed Dataset Limitation:** Cannot adapt to new situations or learn from mistakes
- **Delayed Feedback:** Rewards may come much later than the actions that caused them

Why RL?

RL addresses these challenges by learning policies that maximize expected cumulative rewards, enabling agents to make decisions that consider both immediate and future consequences.

RL Mathematical Framework

RL models decision-making as a Markov Decision Process (MDP), where an agent interacts with an environment through states, actions, and rewards.

Core Mathematical Components:

An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: State space (all possible situations)
- $\mathcal{A}$: Action space (all possible actions)
- $P(s'|s, a)$: Transition probability (probability of reaching state s' from state s after action a)
- $R(s, a)$: Reward function (immediate reward for taking action a in state s)
- $\gamma \in [0, 1]$: Discount factor (importance of future vs. immediate rewards)

Policy and Value Functions:

A policy $\pi(a|s)$ represents the agent's strategy, giving the probability of taking action a in state s . The goal is to find the optimal policy π^* that maximizes expected returns:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \quad (1)$$

Explaining Equation (1):

This equation defines the objective that RL tries to maximize. Here's what each part means:

- $\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)$: The discounted sum of rewards over time. At each timestep t , we get reward $R(s_t, a_t)$, but future rewards are discounted by γ^t .
- $\mathbb{E}_{\pi}[\cdot]$: Expected value under policy π . Since the policy is stochastic (probabilistic), we average over all possible sequences of states and actions.
- γ^t : Discount factor ensures that immediate rewards are weighted more heavily than distant future rewards. If $\gamma = 0.9$, a reward 10 steps in the future is worth only $0.9^{10} \approx 0.35$ times its immediate value.

In simple terms, this equation captures the total expected reward the agent will receive by following policy π , with future rewards discounted appropriately.

The state value function measures expected return from a given state:

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right] \quad (2)$$

The action value function (Q-function) measures expected return from taking a specific action in a specific state:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a \right] \quad (3)$$

Explaining Equations (2) and (3):

These equations break down the total expected return:

- $V^{\pi}(s)$: "How good is it to be in state s under policy π ?" This averages over all possible actions the policy might take from that state.
- $Q^{\pi}(s, a)$: "How good is it to take action a in state s , then follow policy π afterward?" This is more specific, evaluating a particular action choice.

- The relationship: $V^\pi(s) = \sum_a \pi(a|s)Q^\pi(s, a)$ - the state value is the weighted average of action values.

Policy Gradient Methods

Policy gradient methods directly optimize the policy parameters θ to maximize $J(\theta)$. The challenge is computing $\nabla_\theta J(\theta)$ when the objective involves expectations over trajectories that depend on the policy.

Deriving the Policy Gradient Theorem:

Starting with the objective function:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] = \sum_{\tau} P(\tau|\pi_\theta)G(\tau) \quad (4)$$

where $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ represents a trajectory and $G(\tau) = \sum_{t=0}^T \gamma^t r_t$ is its return.

Taking the gradient:

$$\nabla_\theta J(\theta) = \sum_{\tau} \nabla_\theta P(\tau|\pi_\theta)G(\tau) \quad (5)$$

The trajectory probability under policy π_θ is:

$$P(\tau|\pi_\theta) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t)P(s_{t+1}|s_t, a_t) \quad (6)$$

where $\rho_0(s_0)$ is the initial state distribution and $P(s_{t+1}|s_t, a_t)$ are transition probabilities (independent of θ).

Using the log-derivative trick: $\nabla_\theta P(\tau|\pi_\theta) = P(\tau|\pi_\theta)\nabla_\theta \log P(\tau|\pi_\theta)$

Why the Log-Derivative Trick Works:

This identity comes from basic calculus. For any function $f(\theta) > 0$:

$$\frac{d}{d\theta} \log f(\theta) = \frac{1}{f(\theta)} \frac{df(\theta)}{d\theta} \quad (7)$$

$$\text{Rearranging: } \frac{df(\theta)}{d\theta} = f(\theta) \frac{d}{d\theta} \log f(\theta) \quad (8)$$

Applied to our trajectory probability $P(\tau|\pi_\theta)$: $\nabla_\theta P(\tau|\pi_\theta) = P(\tau|\pi_\theta)\nabla_\theta \log P(\tau|\pi_\theta)$

This trick is crucial because:

- It converts a gradient of a product (hard to compute) into a gradient of a sum (easier to compute)
- It allows us to work with log-probabilities, which are more numerically stable
- It enables the expectation form that makes sampling-based estimation possible

$$\nabla_{\theta} J(\theta) = \sum_{\tau} P(\tau|\pi_{\theta}) \nabla_{\theta} \log P(\tau|\pi_{\theta}) G(\tau) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log P(\tau|\pi_{\theta}) G(\tau)] \quad (9)$$

Since only the policy terms depend on θ :

$$\nabla_{\theta} \log P(\tau|\pi_{\theta}) = \nabla_{\theta} \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t|s_t) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \quad (10)$$

Substituting back:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right) G(\tau) \right] \quad (11)$$

This can be rewritten as:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G(\tau) \right] \quad (12)$$

A key insight is that we can replace $G(\tau)$ with G_t (return from timestep t onward) because actions at time t don't affect rewards before time t . This gives us the final policy gradient theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t \right] \quad (13)$$

where $G_t = \sum_{k=t}^T \gamma^{k-t} R(s_k, a_k)$ is the return from timestep t .

Explaining Equation (8):

This fundamental equation shows how to improve the policy:

- $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$: The gradient of log-probability of action a_t in state s_t . This tells us how to change parameters θ to make action a_t more or less likely.
- G_t : The actual return (total discounted reward) obtained from timestep t onward. This acts as a "score" for how good the action was.
- The product: If G_t is high (good return), we want to increase the probability of action a_t ; if low, decrease it.
- $\mathbb{E}_{\pi_{\theta}}[\cdot]$: We average this over many episodes to get a reliable estimate of how to improve the policy.

Why This Works:

The beauty of this derivation is that it transforms an intractable expectation over all possible trajectories into a practical algorithm:

- We don't need to know the environment dynamics $P(s'|s, a)$
- We can estimate the gradient using sample trajectories
- Actions are reinforced proportionally to their contribution to good outcomes

This is the mathematical foundation of the REINFORCE algorithm and leads to more advanced methods like PPO used in RLHF.

RL Pipeline Architecture

The RL learning process is illustrated in Figure 1, showing the agent-environment interaction loop.

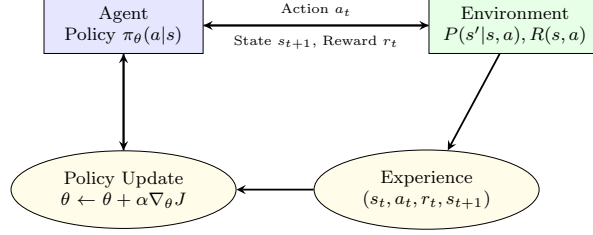


Figure 1: RL agent-environment interaction and learning loop

Worked Example: Simple Grid World Navigation

Let's apply RL to a toy example where an agent learns to navigate a 2×2 grid to reach a goal, illustrating policy gradient learning.

Setup:

- States: $\mathcal{S} = \{(0, 0), (0, 1), (1, 0), (1, 1)\}$ (grid positions)
- Actions: $\mathcal{A} = \{\text{Up, Down, Left, Right}\}$ mapped to indices $[0, 1, 2, 3]$
- Goal: Reach state $(1, 1)$ which gives reward $+10$; all other transitions give reward -1
- Policy: Softmax over linear weights $\theta \in \mathbb{R}^{4 \times 4}$ (4 states $\times$ 4 actions)

Step 1: Initial Policy Parameters

Initialize policy parameters for state $(0, 0)$:

$$\theta_{(0,0)} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \\ 0.0 \end{bmatrix} \quad (14)$$

where each element corresponds to [Up, Down, Left, Right].

Step 2: Compute Action Probabilities

Apply Softmax to get action probabilities in state $(0, 0)$:

$$\pi_{\theta}(a|(0,0)) = \text{Softmax}(\theta_{(0,0)}) = \text{Softmax}\left(\begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \\ 0.0 \end{bmatrix}\right) = \begin{bmatrix} \frac{e^{0.1}}{Z} \\ \frac{e^{-0.2}}{Z} \\ \frac{e^{0.3}}{Z} \\ \frac{e^{0.0}}{Z} \end{bmatrix} \approx \begin{bmatrix} 0.24 \\ 0.18 \\ 0.29 \\ 0.22 \end{bmatrix} \quad (15)$$

where $Z = e^{0.1} + e^{-0.2} + e^{0.3} + e^{0.0} \approx 4.65$.

Initially, "Left" has the highest probability (0.29), but this may not lead to the goal efficiently.

Step 3: Sample Episode

The agent samples actions and experiences one episode:

- $t = 0$: State $(0, 0)$, sample action "Right" (index 3), get reward -1 , move to $(1, 0)$
- $t = 1$: State $(1, 0)$, sample action "Up" (index 0), get reward $+10$, reach goal $(1, 1)$

Episode trajectory: $[(0, 0), 3, -1, (1, 0)] \rightarrow [(1, 0), 0, +10, (1, 1)]$

Step 4: Compute Returns

Calculate discounted returns with $\gamma = 0.9$:

$$G_0 = r_0 + \gamma r_1 = -1 + 0.9 \times 10 = 8.0 \quad (16)$$

$$G_1 = r_1 = 10.0 \quad (17)$$

Step 5: Policy Gradient Update

Update parameters for the first state-action pair ($s_0 = (0, 0)$, $a_0 = 3$):

The gradient of log-probability for action 3 (Right) in state $(0, 0)$:

$$\nabla_{\theta} \log \pi_{\theta}(3|(0, 0)) = \mathbf{e}_3 - \pi_{\theta}(\cdot|(0, 0)) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} - \begin{bmatrix} 0.24 \\ 0.18 \\ 0.29 \\ 0.22 \end{bmatrix} = \begin{bmatrix} -0.24 \\ -0.18 \\ -0.29 \\ 0.78 \end{bmatrix} \quad (18)$$

where $\mathbf{e}_3$ is a one-hot vector for action 3.

Policy gradient update with learning rate $\alpha = 0.1$:

$$\theta'_{(0,0)} = \theta_{(0,0)} + \alpha \cdot G_0 \cdot \nabla_{\theta} \log \pi_{\theta}(3|(0, 0)) \quad (19)$$

$$\theta'_{(0,0)} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \\ 0.0 \end{bmatrix} + 0.1 \times 8.0 \times \begin{bmatrix} -0.24 \\ -0.18 \\ -0.29 \\ 0.78 \end{bmatrix} = \begin{bmatrix} 0.1 - 0.192 \\ -0.2 - 0.144 \\ 0.3 - 0.232 \\ 0.0 + 0.624 \end{bmatrix} = \begin{bmatrix} -0.092 \\ -0.344 \\ 0.068 \\ 0.624 \end{bmatrix} \quad (20)$$

Step 6: Verify Improvement

Compute new action probabilities:

$$\pi'_{\theta}(a|(0, 0)) = \text{Softmax} \left(\begin{bmatrix} -0.092 \\ -0.344 \\ 0.068 \\ 0.624 \end{bmatrix} \right) \approx \begin{bmatrix} 0.17 \\ 0.13 \\ 0.20 \\ 0.35 \end{bmatrix} \quad (21)$$

The probability of taking "Right" (the action that led to the goal) increased from

0.22 to 0.35, while probabilities of other actions decreased. This demonstrates how policy gradient reinforces successful actions.

Step 7: Learning Insight

The positive return $G_0 = 8.0$ reinforced the "Right" action in state $(0, 0)$, making it more likely in future episodes. Over many episodes, the agent will learn to consistently choose actions that lead to higher rewards.

Connection to Language Models

In the context of language models and RLHF:

- **States:** Input prompts or conversation context
- **Actions:** Token generation (choosing next word)
- **Policy:** LLM's probability distribution over vocabulary $\pi_\theta(token|context)$
- **Rewards:** Human feedback scores or preference comparisons

The policy gradient approach enables LLMs to learn from human preferences without requiring explicit labels for every token, making RLHF feasible.

Summary

Reinforcement Learning provides the mathematical foundation for sequential decision-making:

- Models decision problems as MDPs with states, actions, and rewards
- Optimizes policies to maximize expected cumulative rewards
- Uses policy gradients to learn directly from experience
- Enables learning from delayed and sparse feedback

The key insight of RL is using actual outcomes (returns) to reinforce successful actions and discourage unsuccessful ones. This framework becomes crucial for RLHF, where human preferences guide the learning of language models.

Practical Implementation Notes: Real-world RL applications must address:

- **Sample Efficiency** - learning from limited interaction data
- **Exploration vs. Exploitation** - balancing trying new actions vs. using known good ones
- **Credit Assignment** - determining which actions led to rewards
- **Stability** - ensuring consistent learning progress

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.
2. Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4), 229-256.